
A CLEAR, VISUAL INTRODUCTION

Sorting Algorithms

Bubble Sort • Selection Sort • Insertion Sort

2 3 4 5 8

What Is a Sorting Algorithm?

A sorting algorithm is a precise, step-by-step procedure that takes a collection of items and rearranges them into a defined order — typically ascending or descending — based on a comparison rule.

It doesn't just move items around — it repeats the same rule (compare, then place) until every element sits in its correct position relative to its neighbors.



Why Do We Use Sorting Algorithms?

1

Faster Searching

Sorted data lets us use binary search and similar techniques instead of scanning everything.

2

Efficient Processing

Many algorithms (merging, deduplication, ranking) assume or run best on sorted input.

3

Clarity for Users

Ordered lists, leaderboards, and search results are easier to read and act on.

4

A Learning Foundation

Sorting teaches core ideas — comparisons, loops, complexity — used across computer science.

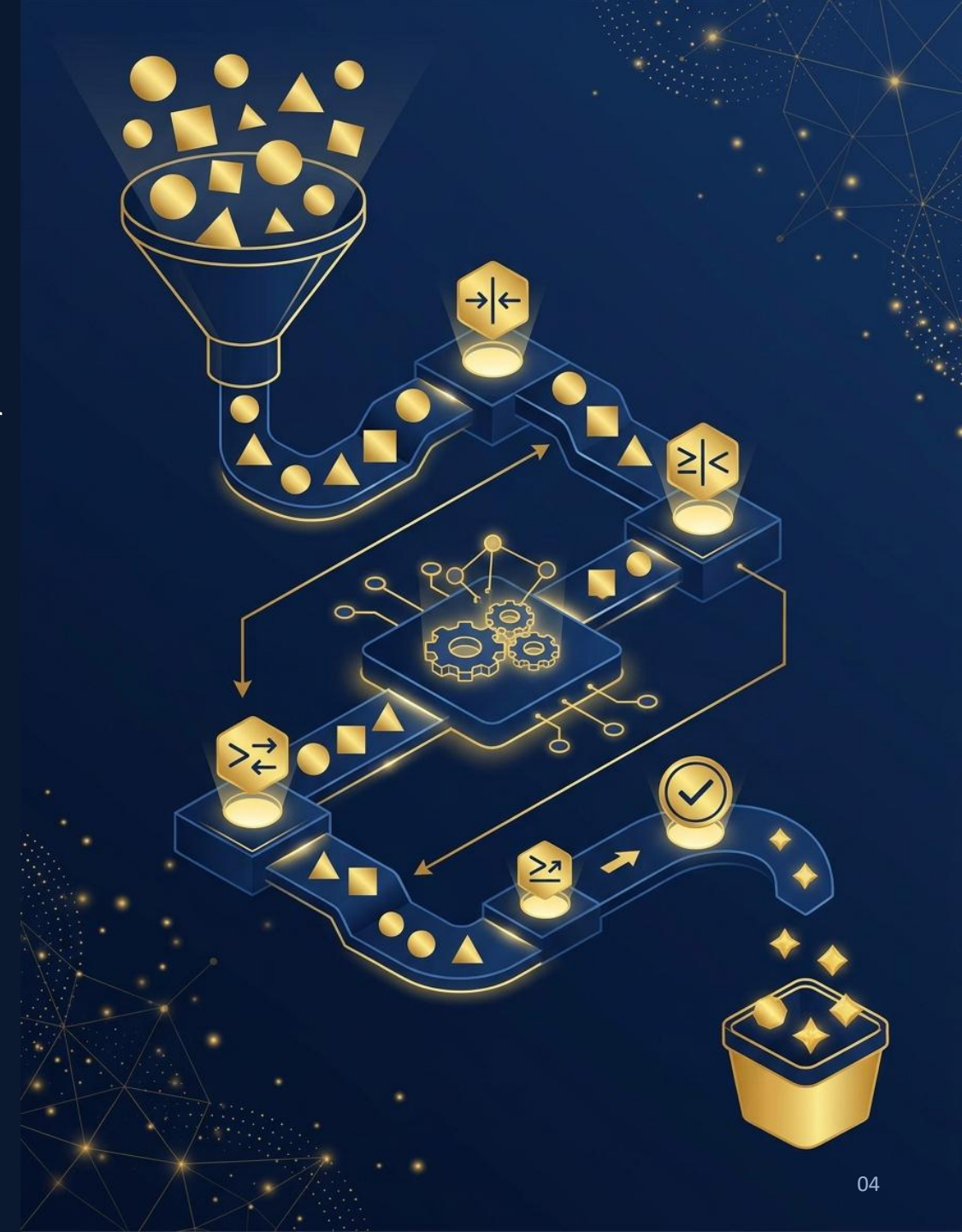
ALGORITHM 01

Bubble Sort — Definition

Bubble Sort repeatedly steps through the list, compares each pair of adjacent elements, and swaps them if they are in the wrong order. Larger values “bubble” toward the end with every pass.

HOW IT WORKS

- 1 Compare each pair of neighboring elements, left to right.
- 2 Swap them if the left one is bigger than the right one.
- 3 Reach the end of the list — the largest value is now in place.
- 4 Repeat for the remaining unsorted portion until no swaps are needed.



Visualizing Bubble Sort

Start
unsorted array



Pass 1
largest bubbles up



Pass 2



Pass 3



Final
fully sorted



 Placed in final position  Still being compared

ALGORITHM 02

Selection Sort — Definition

Selection Sort divides the list into a sorted and an unsorted part. On every pass, it scans the unsorted part to find the smallest value and moves it to the end of the sorted part.

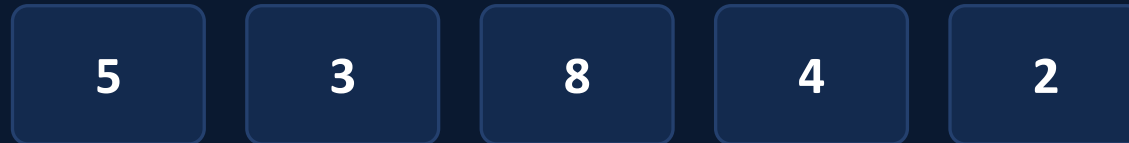
HOW IT WORKS

- 1 Search the unsorted portion for the smallest remaining value.
- 2 Swap it into the next open position at the front.
- 3 Grow the sorted portion by one element.
- 4 Repeat until the whole array is sorted.



Visualizing Selection Sort

Start
unsorted array



Pass 1
smallest → front



Pass 2



Pass 3



Final
fully sorted



 Sorted portion (front)

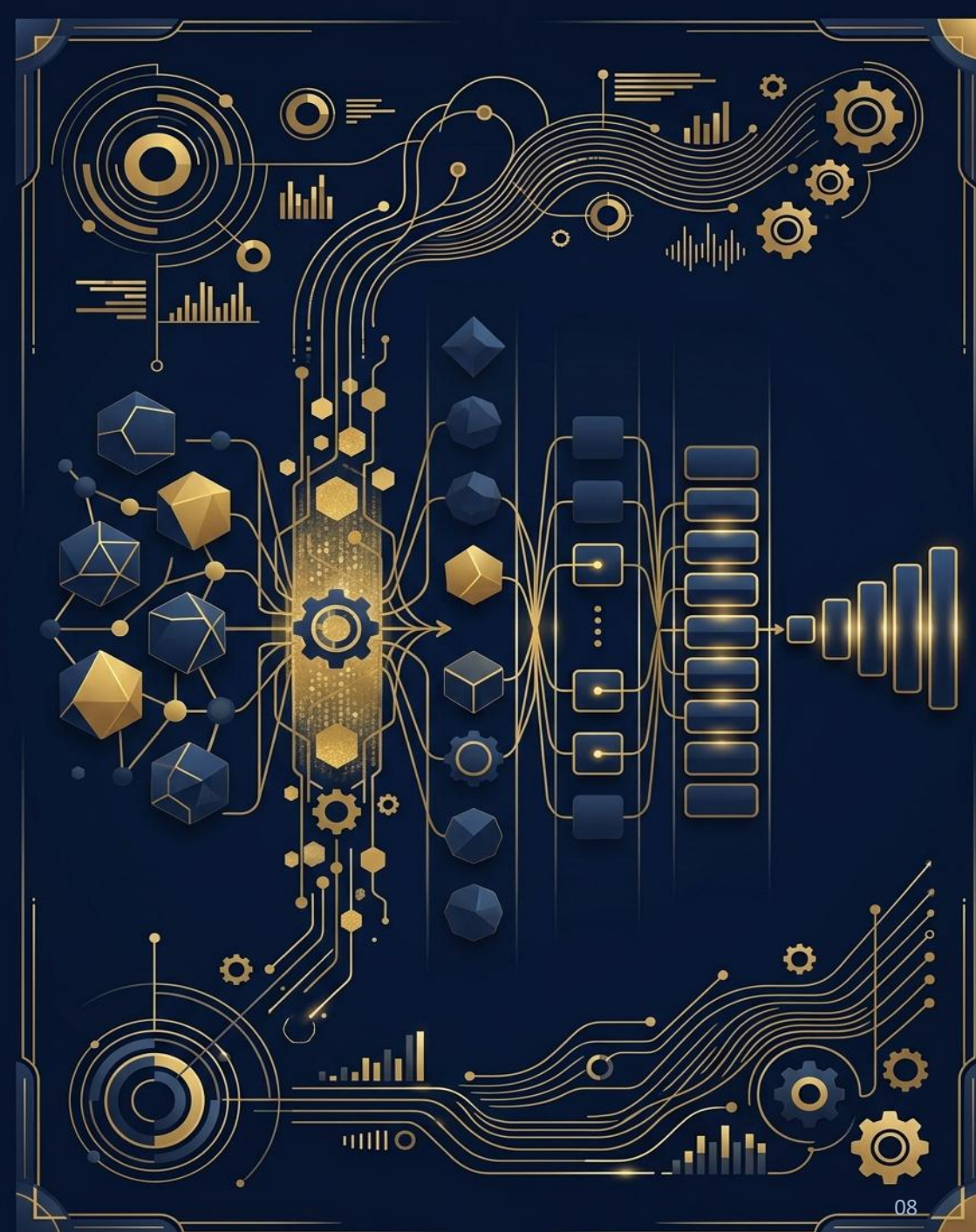
 Unsorted portion

Insertion Sort — Definition

Insertion Sort builds the sorted list one element at a time. It takes the next unsorted value and inserts it into its correct position among the already-sorted elements, shifting others as needed.

HOW IT WORKS

- 1 Treat the first element as a sorted list of one.
- 2 Take the next element and compare it backward through the sorted part.
- 3 Shift larger elements right to make room.
- 4 Insert the element into its correct slot; repeat for all remaining items.



Visualizing Insertion Sort

Start
first element sorted



Insert 3



Insert 8



Insert 4



Insert 2
fully sorted



 Sorted portion (grows left)  Not yet inserted

THANK YOU FOR YOUR ATTENTION

Fin

Questions ?

2 3 4 5 8